Understanding ACT: CVAE and Policy Forward Pass

Explaining the Style Variable Generation

1 Introduction

During the training phase of Action Chunking with Transformers (ACT), the model behaves as a Conditional Variational Autoencoder (CVAE). This means before the main Transformer (the Decoder) can process the vision and proprioception context, we must compute a "style variable," denoted as z. This variable is designed to capture the stochasticity, multi-modality, and specific human nuances of a given demonstration.

This document details the pipeline of the CVAE Encoder up to the point where all arguments are prepared for the main loss function.

2 The CVAE Encoder Inputs

To calculate z during training, the CVAE Encoder is fed a combination of the current state and the actual future actions from the dataset.

In paper terminology, the full observation o_t includes both images and state. The encoder uses $\bar{o}_t$, which is defined as the **observation without the images**. In practice, this means the encoder only sees the low-dimensional physical data. To save on heavy computation (avoiding ResNet passes for the encoder), the images are discarded here.

- **Proprioception ($\bar{o}_t$):** The normalized joint positions of the single-arm state. This is the only part of the observation used by the encoder.
- **Future Actions ($a_{t:t+k}$):** The sequence of normalized k future target joint positions that the human expert actually executed.
- **[CLS] Token:** As described in Section IV.C (page 6) of the original paper, a learnable "Class" token is prepended to the input sequence. Because Transformers output a sequence of the same length as their input, we need a mechanism to condense a list of k actions into a *single* vector. The `[CLS]` token acts as a **Designated Style Extractor**—a dummy variable whose sole purpose is to "listen" to all the other tokens and serve as the final summary bucket.

3 The Encoder Forward Pass

The inputs are mapped to the hidden dimension size (e.g., 512) and passed through a Transformer Encoder. The paper refers to this as "BERT-like." This simply means it is a bidirectional encoder: instead of reading actions sequentially from left to right (like predicting the next word in a sentence), it looks at the entire sequence of future actions all at once to fully understand the context.

This architecture acts as a **Universal Translator**: it reads a "paragraph" of complex human physical movements and translates it into a single "word" (the style vector z). This forces the model to take everything that happened in a long, complicated human demonstration and summarize it into one single spot (the final `[CLS]` token).

3.1 Aggregating Information

Inside this Encoder, multi-head self-attention allow the `[CLS]` token to interact with the entire mathematical sequence of future actions and the proprioception vector. By the time the data passes through all the layers:

- The output vectors corresponding to the individual actions are discarded.
- The single vector located at the `[CLS]` position is extracted. Because of self-attention, this single vector now contains a summarized mathematical representation of the entire human action sequence's "style" in that specific state.

3.2 Predicting the Distribution: The Gaussian Assumption

Instead of generating a static tensor, ACT assumes the "style" of a trajectory is a random process following a **Multivariate Gaussian Distribution**. To solve this, the summary `[CLS]` feature is passed through two linear layers to predict the statistical parameters of that distribution:

1. **Mean Vector (μ):** The center of the "style" distribution in the 512-dimensional space.
2. **Variance (σ^2):** A vector representing the diagonal variance (the "uncertainty" or "spread") of each style dimension.

During training, the model performs **Dual Learning**: it simultaneously refines its ability to "translate" human motion (the Encoder's job, using weights ϕ) while also sharpening its ability to "execute" those movements (the Policy Transformer's job, using weights θ).

4 Sampling and the Reparameterization Trick

Instead of directly passing the mean as z, we sample from the predicted Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$.

To maintain differentiability for backpropagation during training, a concept called the *reparameterization trick* is applied:

$$z = \mu + \sigma \cdot \epsilon$$

Where ϵ is random noise sampled from a standard normal distribution $\mathcal{N}(0, 1)$. This z vector is finally projected to the standard hidden dimension (e.g., 512).

5 The Policy Transformer: Action Generation

Now that z has been sampled, the model moves to its second major component: the Policy Transformer. This is a standard Transformer Encoder-Decoder architecture responsible for physically commanding the robot.

5.1 The Policy Encoder: Building Context

The Policy Encoder takes the normalized sensor data and combines it with the style variable to create a unified understanding of the current "scene." The input sequence is constructed by concatenating:

1. **Vision Tokens:** The $600 - 900$ visual features extracted from the 2-3 cameras (plus 2D positional embeddings).
2. **Proprioception Token:** The single token representing the robot's state.
3. **Style Token:** The projected z vector.

Inside the Policy Encoder, self-attention layers allow all these modalities to interact. The wrist camera features can "contextualize" themselves using the joint angles, while keeping the human's style (z) in mind. The output is a highly enriched set of memory tokens.

5.2 The Policy Decoder: Predicting the Chunk

The Policy Decoder's job is to read the Encoder's memory and output a sequence of k future actions (e.g., $k = 100$).

- **Queries:** Instead of feeding inputs sequentially like a language model, the Decoder is given k fixed positional embeddings. Think of these as 100 empty "slots" asking the Encoder, "What should action 1 be?", "What should action 2 be?", etc.
- **Cross-Attention:** Through cross-attention, these slots query the rich memory from the Policy Encoder, filling themselves with geometric and semantic information.
- **Output ($\hat{a}_{t:t+k}$):** The output of the Decoder is passed through a final linear layer to predict the k normalized, continuous joint actions for the robot.

6 Preparing for the Loss Function

By running both the CVAE Encoder (ϕ) and the Policy Transformer (θ), we have successfully generated every mathematical argument needed to compute the two halves of the total loss:

- **KL Divergence Arguments:** We have the predicted parameters (μ, σ^2) from the CVAE Encoder. These are used to compute the KL loss, ensuring the style distribution doesn't stray too far from a standard normal prior $\mathcal{N}(0, I)$.
- **Reconstruction Arguments:** We now have the Policy Decoder's final predictions ($\hat{a}_{t:t+k}$). We compare these directly to the ground truth human actions ($a_{t:t+k}$) using an L1 smooth reconstruction loss.

Simple Summary: To recap the forward pass, we take the expert actions and joint state to "translate" them into a style distribution (μ, σ^2). We sample a style vector z from this distribution and feed it into the main Policy Transformer alongside the camera images. This produces a predicted chunk of 100 actions. We now have everything we need to calculate how far off our predictions were (L1 Loss) and how messy our translation was (KL Loss).